

```
#include <string.h>
#include <math.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_timer.h"
#include "esp_log.h"
#include "driver/gpio.h"
#include "driver/i2c.h"
#include "esp_bt.h"
#include "esp_bt_main.h"
#include "esp_gap_bt_api.h"
#include "esp_spp_api.h"
#include "nvs_flash.h" static const char *TAG = "REFLEX";
```

```
#define TARGET_SHAKE 0xFF
#define TARGET_NONE 0xFE
#define I2C_MASTER_NUM I2C_NUM_0
#define I2C_MASTER_SDA_IO 21
#define I2C_MASTER_SCL_IO 22
#define I2C_MASTER_FREQ_HZ 400000
#define MPU6050_ADDR 0x68
#define SHAKE_THRESHOLD_G 1.8f
#define START_BYTE 0xAA
#define CMD_START_CHALLENGE 0x01
#define CMD_RESET 0x02
#define RESP_RESULT 0x81
#define RESP_ACK 0x82
#define OUTCOME_CORRECT 0x00
#define OUTCOME_WRONG_BTN 0x01
#define OUTCOME_TIMEOUT 0x02
```

```
#define NUM_BUTTONS 4
static const int buttons_pins[NUM_BUTTONS] = {34,35,36,39}
static const int led_pins[NUM_BUTTONS] = {27, 14, 12, 13};
```

```
static volatile uint8_t current_target = 0xFE;
static int64_t challenge_start_us = 0;
static volatile bool round_active = false;
static uint32_t spp_handle = 0;
```

```
static QueueHandle_t gpio_evt_queue = NULL;
```

```
static void start_round_timer(void) {
    int64_t time_us = esp_timer_get_time();
    Round_active = true;
}
```

```

static uint16_t elapsed_ms(void) {
    int64_t time_us = esp_timer_get_time() - challenge_start_us;
    round_active = false;
    return (uint16_t)(time_us / 1000);
}

//SPP

static void send_frame(uint8_t msg_type, uint8_t *payload, uint8_t payload_len) {
    uint8_t frame[16];
    uint8_t idx = 0;
    frame[idx++] = START_BYTE;
    frame[idx++] = msg_type;
    uint8_t checksum = msg_type;
    for (int i = 0; i < payload_len; i++) {
        frame[idx++] = payload[i];
        checksum ^= payload[i];
    }
    frame[idx++] = checksum;

    if (spp_handle != 0) {
        esp_spp_write(spp_handle, idx, frame);
    }
}

static void send_result(uint8_t outcome) {
    uint16_t t = elapsed_ms();
    uint8_t payload[3] = { outcome, (uint8_t)(t & 0xFF), (uint8_t)((t >> 8) & 0xFF) };
    send_frame(ESP_RESULT, payload, 3);
}

static void send_ack(uint8_t target_id) {
    send_frame(ESP_ACK, &target_id, 1);
}

static void clear_leds(void) {
    for (int i = 0; i < NUM_BUTTONS; i++) gpio_set_level(led_pins[i], 0);
}

static void end_round(void) {
    round_active = false;
    current_target = TARGET_NONE;
    clear_leds();
    if (timeout_timer) esp_timer_stop(timeout_timer);
}

static void begin_challenge(uint8_t target, uint16_t timeout_ms) {
    clear_leds();

```

```

    current_target = target;
    if (target < NUM_BUTTONS) {
        gpio_set_level(led_pins[target], 1);
    }
    start_round_timer();
    send_ack(target);

    if (timeout_timer) {
        esp_timer_stop(timeout_timer);
        esp_timer_start_once(timeout_timer, (uint64_t)timeout_ms * 1000ULL);
    }
}

```

```

static void timeout_cb(void * arg) {
    if (round_active) {
        send_result(OUTCOME_TIMEOUT);
        end_round();
    }
}

```

//buttons

```

static void IRAM_ATTR button_isr_handler(void *arg) {
    uint32_t idx = (uint32_t)arg;
    xQueueSendFromISR(gpio_evt_queue, &idx, NULL);
}

```

```

static void button_task(void *arg) {
    uint32_t idx;
    int64_t last_press_us[NUM_BUTTONS] = {0};

    while(1) {
        if (xQueueReceive(gpio_evt_queue, &idx, portMAX_DELAY)) {
            int64_t now = esp_timer_get_time();
            if (now - last_press_us[idx] < 50000) continue;
            last_press_us[idx] = now;

            if (!round_active) continue;
            if (current_target == TARGET_SHAKE) {
                send_result(OUTCOME_WRONG_BTN);
            } else if (idx == current_target) {
                send_result(OUTCOME_CORRECT);
            } else {
                send_result(OUTCOME_WRONG_BTN);
            }
            end_round();
        }
    }
}

```

```

    }
}

static void setup_buttons_and_leds(void) {
    gpio_evt_queue = xQueueCreate(10, sizeof(uint32_t));
    for (int i = 0; i < NUM_BUTTONS; i++) {
        gpio_config_t led_conf = {
            .pin_bit_mask = (1ULL << led_pins[i]),
            .mode = GPIO_MODE_OUTPUT,
        };
        gpio_config(&led_conf);
        gpio_set_level(led_pin[i], 0);

        gpio_config_t btn_conf = {
            .pin_bit_mask = (1ULL << button_pins[i]),
            .mode = GPIO_MODE_INPUT,
            .pull_up_en = GPIO_PULLUP_ENABLE,
            .intr_type = GPIO_INTR_NEGEDGE,
        };
        gpio_config(&btn_conf);
    }

    gpio_install_isr_service(0);
    for (int i = 0; i < NUM_BUTTONS; i++) {
        gpio_isr_handler_add(button_pins[i], button_isr_handler, (void *) (uint32_t) i);
    }

    xTaskCreate(button_task, "button_task", 2048, NULL, 10, NULL);
}

static esp_err_t imu_read_accel(float *ax, float *ay, float *az) {
    uint8_t data[6];
    uint8_t reg = 0x3B; //assuming we using a MPU6050
    esp_err_t ret = i2c_master_write_read_device(I2C_MASTER_NUM,
        MPU6050_ADDR, &reg, 1, data, 6, pdMS_TO_TICKS(50));
    if(ret != ESP_OK) return ret;

    int16_t raw_x = (data[0] << 8) | data[1];
    int16_t raw_y = (data[2] << 8) | data[3];
    int16_t raw_z = (data[4] << 8) | data[5];

    *ax = raw_x/16384.0f;
    *ay = raw_y/16384.0f;
    *az = raw_z/16384.0f;
    return ESP_OK;
}

```

```

static void imu_task(void *arg) {
    while (1) {
        if (round_active && current_target == TARGET_SHAKE) {
            float ax, ay, az;
            if (imu_read_accel(&ax, &ay, &az) == ESP_OK) {
                float mag = sqrtf(ax * ax + ay * ay + az * az);
                If (mag > SHAKE_THRESHOLD_G) {
                    send_result(OUTCOME_CORRECT);
                    end_round();
                }
            }
        }
        vTaskDelay(pdMS_TO_TICKS(10));
    }
}

```

```

static void setup_i2c(void) {
    i2c_config_t conf = {
        .mode = I2C_MODE_MASTER,
        .sda_io_num = I2C_MASTER_SDA_IO,
        .scl_io_num = I2C_MASTER_SCL_IO,
        .sda_pullup_en = GPIO_PULLUP_ENABLE,
        .scl_pullup_en = GPIO_PULLUP_ENABLE,
        .master.clk_speed = I2C_MASTER_FREQ_HZ,
    };
    i2c_param_config(I2C_MASTER_NUM, &conf);
    i2c_driver_install(I2C_MASTER_NUM, conf.mode, 0, 0, 0);

    uint8_t wake_cmd[2] = {0x6B, 0x00};
    i2c_master_write_to_device(I2C_MASTER_NUM, MPU6050_ADDR, wake_cmd, 2,
pdMS_TO_TICKS(100) );
}

```

```

static void handle_incoming_command(uint8_t *data, uint16_t len) {
    if (len < 3 || data[0] != START_BYTE) return;

    uint8_t msg_type = data[1];

    switch (msg_type) {
        case CMD_START_CHALLENGE: {
            if (len < 6) return;
            uint8_t target = data[2];
            uint16_t timeout_ms = data[3] | (data[4] << 8 );
            break;
        }
        case CMD_RESET:

```

```

        end_round();
        break;
    default:
        ESP_LOGW(TAG, "Unknown command type goober: 0x%02X",
msg_type);
        break;
    }
}

```

```

static void esp_spp_cb(esp_spp_cb_event_t event, esp_spp_cb_param_t *param) {
    switch (event) {
        case ESP_SPP_INIT_EVT:
            esp_bt_dev_set_device_name("Reflex Trainer");
            esp_bt_gap_set_scan_mode(ESP_BT_CONNECTABLE,
ESP_BT_GENERAL_DISCOVERABLE);
            esp_spp_start_srv(ESP_SPP_SEC_NONE,
ESP_SPP_ROLE_SLAVE, 0, "REFLEX_SPP");
            break;
        case ESP_SPP_SRV_OPEN_EVT:
            spp_handle = param->srv_open.handle;
            ESP_LOGI(TAG, "Client connected, handle=%lu", (unsigned
long)spp_handle);
            break;
        case ESP_SPP_DATA_IND_EVT:
            handle_incoming_command(param->data_ind.data,
param->data_ind.len);
            break;
        case ESP_SPP_CLOSE_EVT:
            ESP_LOGI(TAG, "Client disconnected :(");
            spp_handle = 0;
            end_round();
            break;
        default:
            break;
    }
}

```

```

void app_main(void) {
    esp_err_t ret = nvs_flash_init();
    if (ret == ESP_ERR_NVS_NO_FREE_PAGES || ret ==
ESP_ERR_NVS_NEW_VERSION_FOUND) {
        ESP_ERROR_CHECK(nvs_flash_erase());
        ret = nvs_flash_init();
    }
    ESP_ERROR_CHECK(ret);
}

```

```

setup_buttons_and_leds();
setup_i2c();

esp_bt_controller_config_t bt_cfg = BT_CONTROLLER_INIT_CONFIG_DEFAULT();
ESP_ERROR_CHECK(esp_bt_controller_init(&bt_cfg));
ESP_ERROR_CHECK(esp_bt_controller_enable(ESP_BT_MODE_CLASSIC_BT));

ESP_ERROR_CHECK(esp_bluedroid_init());
ESP_ERROR_CHECK(esp_bluedroid_enable());

ESP_ERROR_CHECK(esp_spp_register_callback(esp_spp_cb));
esp_spp_cfg_t spp_cfg = BT_SPP_DEFAULT_CONFIG();
ESP_ERROR_CHECK(esp_spp_enhanced_init(&spp_cfg));

const esp_timer_create_args_t timer_args = {
    .callback = &timeout_cb,
    .name = "round_timeout"
};
ESP_ERROR_CHECK(esp_timer_create(&timer_args, &timeout_timer));

xTaskCreate(imu_task, "imu_task", 2048, NULL, 5, NULL);

ESP_LOGI(TAG, "Reflex Trainer ready and advertising as SPP device.");
}

```